



REALTEK

Software Development Kit

Switch Product

Linux Kernel 2.6.32.58

Porting Guide

[include GCC and uClibc upgraded]

(CONFIDENTIAL: Development Partners Only)

Rev. 1.0.7

15 September 2014



Realtek Semiconductor Corp.

No. 2, Innovation Road II, Hsinchu Science Park, Hsinchu 300, Taiwan

Tel.: +886-3-578-0211 Fax: +886-3-577-6047

www.realtek.com

COPYRIGHT

© 2008-2011 Realtek Semiconductor Corp. All rights reserved. No part of this document may be reproduced, transmitted, transcribed, stored in a retrieval system, or translated into any language in any form or by any means without the written permission of Realtek Semiconductor Corp.

TRADEMARKS

Realtek is a trademark of Realtek Semiconductor Corporation. Other names mentioned in this document are trademarks/registered trademarks of their respective owners.

DISCLAIMER

Realtek provides this document “as is”, without warranty of any kind, neither expressed nor implied, including, but not limited to, the particular purpose. Realtek may make improvements and/or changes in this document or in the product described in this document at any time. This document could include technical inaccuracies or typographical errors.

USING THIS DOCUMENT

This document is intended for use by the system engineer when integrating with Realtek Switch Software SDK. Though every effort has been made to assure that this document is current and accurate, more information may have become available subsequent to the production of this guide. In that event, please contact your Realtek representative for additional information that may help in the development process.

REVISION HISTORY

Revision	Release Date	Summary
1.0.0	2012/04/19	Init draft
1.0.1	2012/04/30	1. Modify the wording. 2. Add Makefile modification items. 3. Add the U-Boot Makefile modification section.
1.0.2	2012/05/02	1. Update msdk version. 2. Change the layout content.
1.0.3	2012/05/03	1. Modify the wording.
1.0.4	2012/05/15	1. Modify the section 5.1 description. 2. Add the chapter 6.
1.0.5	2012/07/17	1. Modify the description for –mmt in section 1.1. 2. Add the description for MIPS 34Kc in section 4.1 3. Add the description for MIPS 34Kc in section 5.3
1.0.5.1	2013/04/17	1. Modify the section 2.1.1 and 2.1.2 description

Revision	Release Date	Summary
1.0.6	2013/04/25	1. Updated section 2.2.1
1.0.7	2014/09/15	1. Updated section 2.1 description. 2. Updated section 2.1.3 description. 3. Updated section 4.3 description.

Table of Contents

Chapter 1	Introduction	6
1.1.	msdk-4.3.6-mips-EB-2.6.32-0.9.30.3-m32-120424	6
1.2.	Linux Kernel 2.6.32.58	7
1.3.	GCC 4.3.6 and uClibc 0.9.30.3	7
Chapter 2	Porting SDK modules into Kernel	8
2.1	SDK Kernel Module Porting	8
	2.1.1 /net folder	8
	2.1.2 /drivers/net folder	9
	2.1.3 /drivers/mtd/maps folder	10
2.2	SDK Linux Kernel BSP Porting	11
	2.2.1 /linux-2.6.32.x/arch/mips/bsp folder	11
	2.2.2 Platform Header Files	12
	2.2.3 Platform Init Module	12
	2.2.4 Timer Module	12
	2.2.5 UART Module	12
	2.2.6 IRQ Module	13
Chapter 3	Porting with uClinux architecture	14
3.1	New Kernel folder	14
3.2	New uClibc folder	14
3.3	SDK source code folder	15
3.4	Configuration File	16
Chapter 4	Make Flow Modification	17
4.1	config.arch	17
4.2	rc	17
4.3	Makefile in /uClinux folder	18
4.4	Makefile in /lib folder	19
4.5	Pre-Created Symbolic Links	19
4.6	SDK Kernel Module Makefile	20
4.7	SDK Source code	20
Chapter 5	U-Boot Makefile Modification	21
5.1	config.mk in /u-boot/cpu/mips/ folder	21
5.2	config.mk in /u-boot/ folder	21
5.3	Makefile in /u-boot/ folder	21
Chapter 6	User Folder Makefile Modification	22

6.1	Makefile in /kernel/uClinux/user/busybox/	22
6.2	Makefile in /kernel/uClinux/user/switch/	22

Chapter 1 Introduction

This document is used to introduce how to upgrade the Linux kernel to 2.6.32.58 from 2.6.19. And corresponding, upgrade procedure GCC to 4.3.6 and the uClibc to 0.9.30.3.

For another approach is to porting Realtek SDK driver along to existing linux platform environment (with existing kernel, toolchain). In this case, chapter 2 will help to guide to moving SDK modules and BSP into your existing Linux software platform.

The following is used version of toolchains for this upgrade program.

1.1. msdk-4.3.6-mips-EB-2.6.32-0.9.30.3-m32-120

424

This is tool chain's naming rule.

msdk	: For MIPS CPU core
4.3.6	: GCC version
EB	: Big Endian
2.6.32	: Linux kernel version
0.9.30.3	: uClibc version
m32	: mips 32bit-library wrapper
120424	: release date.

Once the MSDK package is decompressed, the **msdk-4.3.6-mips-EB-2.6.32-0.9.30.3-m32-120424** folder will appear. This folder contains several folders which are shown as following.

- ◆ bin
- ◆ config
- ◆ include
- ◆ lib
- ◆ libexec
- ◆ mips-linux

Please set up \$PATH in shell environment, and add the **msdk-4.3.6-mips-EB-2.6.32-0.9.30.3-m32-120424/bin** folder in it, then msdk tools and binary utilities could be used.

Once the compile will use for MIPS34Kc, the “-mmt” should be added in CFLAGS.

1.2. Linux Kernel 2.6.32.58

The original source codes came from MIPS official website and there are some code size optimization and create the uniform RTK BSP API made by Realtek.

1.3. GCC 4.3.6 and uClibc 0.9.30.3

Realtek did some package efforts for tool chain, and try to let user can use these tools more easily.

Chapter 2 Porting SDK modules into Kernel

This chapter introduced how to integrate Realtek SDK kernel modules into the Linux kernel.

2.1 SDK Kernel Module Porting

Without BSP parts, the SDK kernel module is separated into three parts.

CONFIG_SDK_DRIVER_COMPILE is stored in /sdk/config/.config which is generated after make menuconfig. By SDK default configuration, the CONFIG_SDK_DRIVER_COMPILE=m, once customer doesn't use Realtek's building system, customer can replace the CONFIG_SDK_DRIVER_COMPILE to 'm' in Makefile directly.

2.1.1 /net folder

After menuconfig, the switch folder will be created over here automatically, and two symbolic links will be created under /net/switch folder which will link to corresponded file by menuconfig. Those corresponded files locate in SDK source code folder.

The following two symbolic links will be created in /net/switch folder.

- 1 Makefile (link to *Makefile.linux.net*)
- 2 rtdrv

The related source codes of /rtdrv folder are located in PRJDIR/sdk/system/linux/rtdrv folder. In this /rtdrv folder, the Makefile includes the *Makefile.linux.kernel.rtdrv*.

For this part, the original kernel /net/Makefile has to append SDK related items only.

In the Figure-2.1.1-1, the original Makefile is end in the RED line, the SDK related items are appended in the end of the original Makefile.

In this time, the following three items are added.

```
obj-$(CONFIG_SDK_DRIVER_COMPILE) += switch/
obj-$(CONFIG_SYS_PROTO_L2G) += switch/
obj-$(CONFIG_SYS_PLAT_LXVLAN) += switch/net/core/vlan/
```



```

ifeq ($(CONFIG_NET),y)
obj-$(CONFIG_SYSCTL) += sysctl_net.o
endif
obj-$(CONFIG_WIMAX) += wimax/

# RTK Software Directory
obj-$(CONFIG_SDK_DRIVER_COMPILE) += switch/
obj-$(CONFIG_SYS_PROTO_L2G) += switch/
obj-$(CONFIG_SYS_PLAT_LXVLAN) += switch/net/core/vlan/

```

Figure 2.1.1-1

2.1.2 /drivers/net folder

After menuconfig, the switch folder will be created over here automatically, and four symbolic links will be created which will link to corresponded file by menuconfig. Those corresponded files locate in SDK source code folder.

The following four symbolic links will be created in /drivers/net/switch folder.

1 Makefile (link to *Makefile.linux.drivers*)

2 rtcore

The related source codes of /rtcore folder are located in PRJDIR/sdk/system/linux/rtcore folder and dev_config.c locates in PRJDIR/sdk/system/linux/ folder. In this folder, the Makefile includes the *Makefile.linux.kernel.rtcore*.

The rtcore module includes dev_config.c in /PRJDIR/sdk/system/linux folder. This file records all Linux device information which includes IRQ number for each device in this chip.

3 rtk

The related source codes of /rtk folder are located in PRJDIR/sdk/system/linux/rtk folder. In this folder, the Makefile includes the *Makefile.linux.kernel.rtk*.

4 rtnic

The related source codes of /rtnic folder are located in PRJDIR/sdk/system/linux/rtnic folder. In this folder, the Makefile includes *Makefile.linux.kernel.rtnic*.

For this part, the original kernel /drivers/net/Makefile has to add one SDK related items only.

As shown in Figure-2.1.2-1, the following is added in Makefile.

```
obj-$(CONFIG_SDK_DRIVER_COMPILE) += switch/
```

```
obj-$(CONFIG_MDIO) += mdio.o
obj-$(CONFIG_PHYLIB) += phy/

obj-$(CONFIG_TI_DAVINCI_EMAC) += davinci_emac.o

obj-$(CONFIG_SDK_DRIVER_COMPILE) += switch/
obj-$(CONFIG_E1000) += e1000/
obj-$(CONFIG_E1000E) += e1000e/
obj-$(CONFIG_IBM_NEW_EMAC) += ibm_newemac/
obj-$(CONFIG_IGB) += igb/
obj-$(CONFIG_IGBVF) += igbvf/
obj-$(CONFIG_IXGBE) += ixgbe/
obj-$(CONFIG_IXGB) += ixgb/
obj-$(CONFIG_IP1000) += ipg.o
obj-$(CONFIG_CHELSIO_T1) += chelsio/
obj-$(CONFIG_CHELSIO_T3) += cxgb3/
obj-$(CONFIG_EHEA) += ehea/
obj-$(CONFIG_CAN) += can/
```

Figure 2.1.2-1

2.1.3 /drivers/mtd/maps folder

After menuconfig, there three symbolic links will be creates which will link to corresponded file by menuconfig. Those corresponded files locate in SDK source code folder.

CONFIG_SDK_ prefix MACRO are stored in /sdk/config/.config which is generated after make menuconfig. By SDK default configuration, the CONFIG_SDK_BSP_MTD_***=y, once customer doesn't use Realtek's building system, customer can replace the CONFIG_SDK_BSP_MTD_*** to 'y' in Makefile directly.

The following three symbolic links will be created.

- 1 rtk_spi_flash_sio.c
- 2 rtk_spi_flash_mio.c
- 3 rtk_nor_flash.c

For this part, the original kernel Makefile has to append SDK related items.

In the Figure-2.1.3-1, the original Makefile is end in the RED line, the SDK related items are appended in the end of the original Makfile.

In this time, the following three items are added.

```
obj-$(CONFIG_SDK_BSP_MTD_SPIS) += rtk_spi_flash_sio.o
obj-$( CONFIG_SDK_BSP_MTD_SPIM) += rtk_spi_flash_mio.o
obj-$( CONFIG_SDK_BSP_MTD_NOR) += rtk_nor_flash.o
```

```
obj-$(CONFIG_MTD_SHAR_NOR) += shar_nor.o
obj-$(CONFIG_MTD_INTEL_VR_NOR) += intel_vr_nor.o
obj-$(CONFIG_MTD_BFIN_ASYNC) += bfin-async-flash.o
obj-$(CONFIG_MTD_RBTX4939) += rbt4939-flash.o
obj-$(CONFIG_MTD_VMU) += vmu-flash.o
obj-$(CONFIG_MTD_GPIO_ADDR) += gpio-addr-flash.o
obj-$(CONFIG_SDK_BSP_MTD_SPIS) += rtk_spi_flash_sio.o
obj-$(CONFIG_SDK_BSP_MTD_SPIM) += rtk_spi_flash_mio.o
obj-$(CONFIG_SDK_BSP_MTD_NOR) += rtk_nor_flash.o
```

Figure 2.1.3-1

2.2 SDK Linux Kernel BSP Porting

Linux BSP APIs are repackaged by Realtek. In the following, the related APIs are introduced in this section.

2.2.1 /linux-2.6.32.x/arch/mips/bsp folder

In fact, this is a symbolic link which will link to real chip BSP folder. The following is BSP symbolic link relationship.

- ```
=====
1. PRJDIR/kernel/uClinux/linux-2.6.32.x/arch/mips/bsp -> ../../target/bsp
2. PRJDIR/kernel/uClinux/target
 -> ../../sdk/system/linux/linux-2.6.32.x/arch/realtek/sel_chip
3. PRJDIR/ sdk/system/linux/linux-2.6.32.x/arch/realtek/sel_chip
 -> ./board/rtl83xx-universal-bsp/
4. PRJDIR/ sdk/system/linux/linux-2.6.32.x/arch/realtek/board/
 rtl83xx-universal-bsp /bsp -> selection/bsp
5. PRJDIR/ sdk/system/linux/linux-2.6.32.x/arch/realtek/board/
 rtl83xx-universal-bsp /selection -> model/mips32r2/
=====
```

Due to consider the model combination, so the symbolic topology is complex, the following example can make this topology is easy to understand.

PRJDIR/kernel/uClinux/linux-2.6.32.x/arch/mips/bsp ->

PRJDIR/sdk/system/linux/linux-2.6.32.x/arch/realtek/board/ rtl83xx-universal-bsp  
/model/ mips32r2/bsp.

Once this Linux kernel needs to support new chip, the new chip's BSP folder should be created in PRJDIR/ sdk/system/linux/linux-2.6.32.x/arch/realtek/board/ folder.

The new chip folder architecture can refer the rtl83xx-universal-bsp folder.

### 2.2.2 Platform Header Files

- `bspcpu.h`  
This header file defines CPU specific configuration, such as, the number of TLB entries, the size as well as line size of ICACHE and DCACHE.
- `bspchip.h`  
This header file defines board specific configuration at the chip level. For example, the I/O mapping address, clock frequency and IRQ assignment...etc.

### 2.2.3 Platform Init Module

This module includes `prom.c` and `setup.c`.

- `bsp_init()`:  
This function does the early platform initialization. Usually, this is the place to set memory range and command line arguments. This function is called by `setup_arch()` in `linux-2.6.32.x/arch/mips/kernel/setup.c`.
- `bsp_free_prom_memory()`:  
This function frees the memory that is occupied by the init module after the initialization process has been completed.
- `bsp_setup()`:  
This function does most of the platform initialization. This function is called by `arch_mem_init()` in `linux-2.6.32.x/arch/mips/kernel/setup.c`.
- `get_system_type()`:  
This function returns the name of the platform. This API is called from several places in the kernel to get identity of the platform.

### 2.2.4 Timer Module

This module is implemented by `timer.c`.

- `bsp_timer_init()`:  
This function does most of the timer initialization. This function is called by `time_init()` in `linux-2.6.32.x/arch/mips/kernel/time.c`.
- `bsp_timer_ack()`:  
This function does the timer interrupt acknowledgement. This function is called by the timer interrupt handler to clear the current interrupt. The actual implementation depends on the interrupt controller handler.

### 2.2.5 UART Module

This module is implemented by `serial.c`.

- `bsp_serial_init()`:

This function does serial port initialization. This function is called by `bsp_setup()` in `setup.c`.

### 2.2.6 IRQ Module

This module is implemented by `irq.c`.

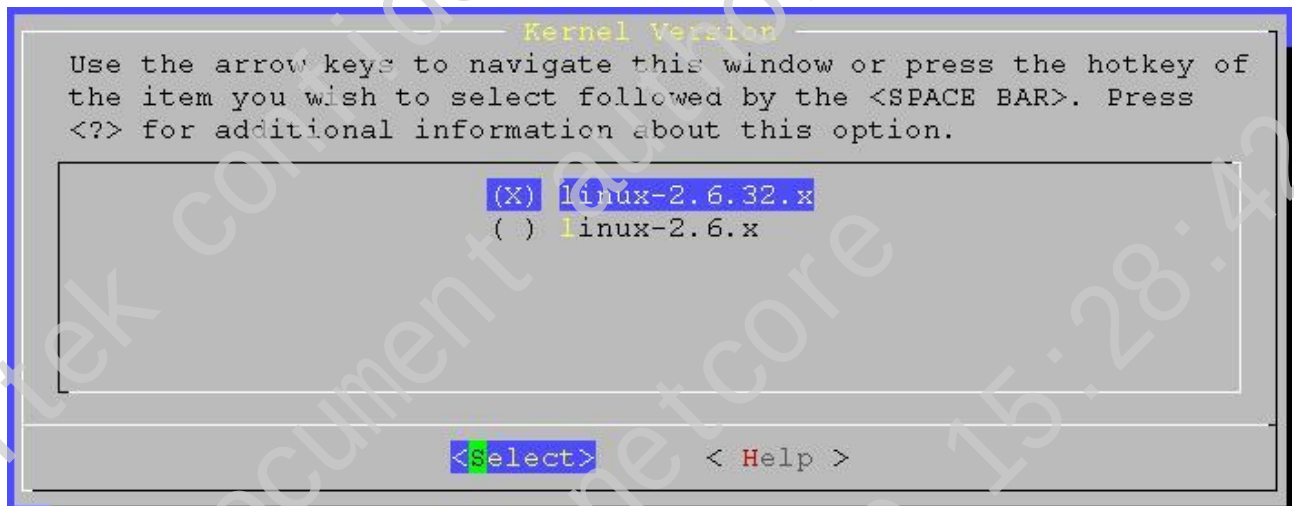
- `bsp_irq_init()`:  
This function does the IRQ initialization for platform. This function is called by `init_IRQ()` in `linux-2.6.32.x/arch/mips/kernel/irq.c`.
- `bsp_irq_dispatch()`:  
This function does the IRQ dispatch for the platform. This function is called by exception handler when interrupt occurs. Interrupts are dispatched to their action handlers accordingly.

## Chapter 3 Porting with uClinux architecture

Once the Realtek uClinux architecture is used already, and wants to add the new kernel, uClibc only, please take care from chapter 3 to chapter 5.

### 3.1 New Kernel folder

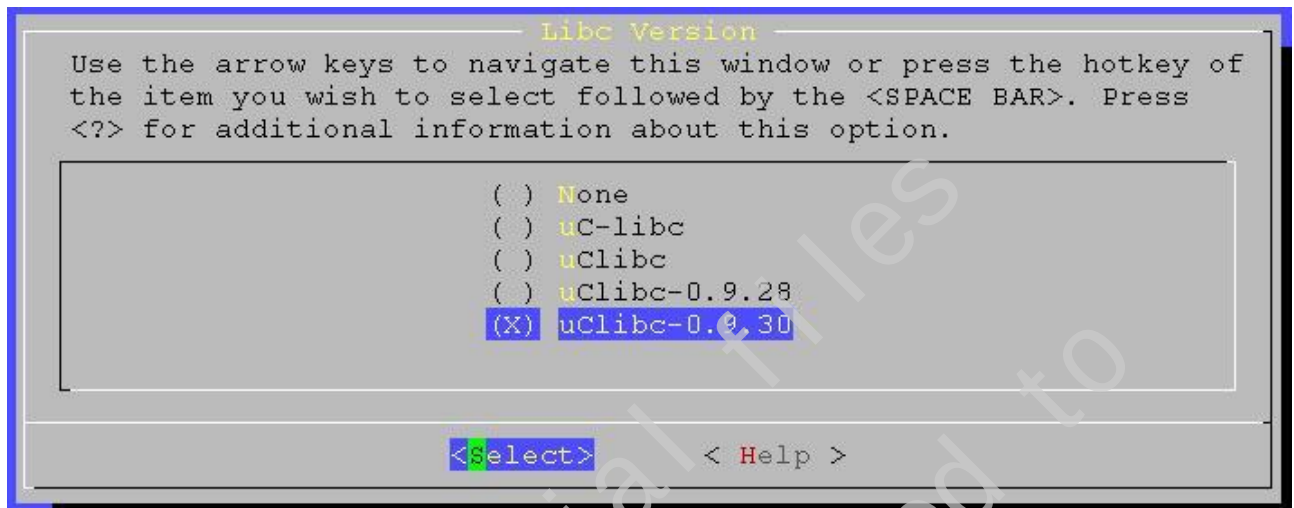
The kernel source tree should be placed in PRJDIR/kernel/uClinux/ folder. Current menuconfig will parser Linux source tree folder name and add it into menu automatically, and the folder name should use the prefix as “linux-“ and the postfix as “.x”. Therefore, the new kernel source tree folder name is “linux-2.6.32.x”, and the related item will appear in menu as follow.



### 3.2 New uClibc folder

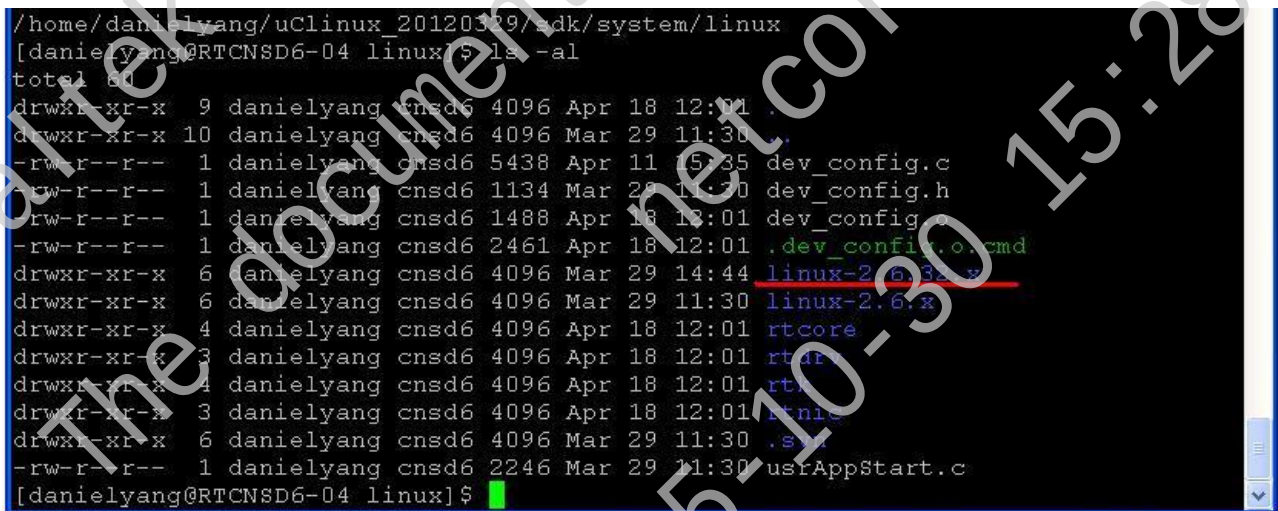
The uClibc should be placed in PRJDIR /kernel/uClinux/ folder. Current menuconfig will parser uClibc folder name, then add it into menu automatically, and the folder name should use the prefix as “uClibc-“. Therefore, the new uClibc folder name is “uClibc-0.9.30”, and the related item will appear in menu as following.





### 3.3 SDK source code folder

The Linux related source codes of SDK are placed in PRJDIR/sdk/system/linux/ folder which include the rtk, rtcore, rtnic and Linux BSP parts. The new kernel related BSP source code will be placed in “linux-2.6.32.x” folder as following.



Under this folder, the /arch folder which depends on BSP parts, due to Realtek did some architecture change, so the placement is different with official distribution a little, other parts are the same.

### 3.4 Configuration File

Basically, there are two configuration files should be added only.

- (1) `config.linux-2.6.32.x` : configuration file for Linux 2.6.32.58 kernel.
- (2) `config.uClinux-0.9.30` : even the uClibc-0.9.30 will not be rebuilt every time, but the default used configuration is kept over here.



## Chapter 4 Make Flow Modification

Basically, the original Linux code base and tool chain (includes GCC and uClibc) came from open domain and ITRI.

But now, the newest Linux code base and tool chain are used, therefore, there are some different compile options and make behaviors should be integrated and modified.

### 4.1 config.arch

- (1) The [1] PRJDIR/kernel/uClinux/.config and [2] kernel .config should be included. Because the version information will be used for modified the make flow and CFLAGS...etc.
- (2) CROSS\_COMPILE and CROSS\_KERNEL\_COMPILE are used as tool chain binary ELF name's prefix.
  - (2).1. Old prefix : mips-linux-uclibc-
  - (2).2. New prefix : msdk-linux-
- (3) For msdk tool chain, the "-nostdinc" should be removed from CFLAGS, then msdk include files could be referenced during make code process.
- (4) Remove the CRTI and CRTN assignment for msdk tool chain.
- (5) Remove the following two items for msdk tool chain
  - (5).1. LDFLAGS += \$(CRTI) \$(LDPATH)
  - (5).2. LDLIBS = -nodefaultlibs \$(LIBC) \$(CRTN)
- (6) Once the MIPS34Kc chip is selected such as RTL8390, and the msdk tool chain is used, then the "-mmt" should be added in CFLAGS.

### 4.2 rc

This file will be installed into Root file system and run it during initial flow on device as well known.

Due to different kernel, the related loadable kernel modules install path will include kernel version number, so the rc file has to use different loadable kernel module path.

Therefore, the original path is "/lib/modules/2.6.19/kernel", but now, the "2.6.19" is changed to "X.X.XX", then the "X.X.XX" string will be modified during romfs install.

Base on configuration file, the "X.X.XX" string in rc file will be modified to

“2.6.19” or “2.6.32.58” in rc file, then install to romfs folder.

### 4.3 Makefile in /uClinux folder

There are some targets and include files are modified in this Makefile. Basically, the menuconfig will follow the configuration of “Product” item to decide the Turnkey configuration should be included or not. Makefile should used the \$(CONFIG\_PRODUCT) to modify the reference rules, such as from (3) to (10) for this purpose.

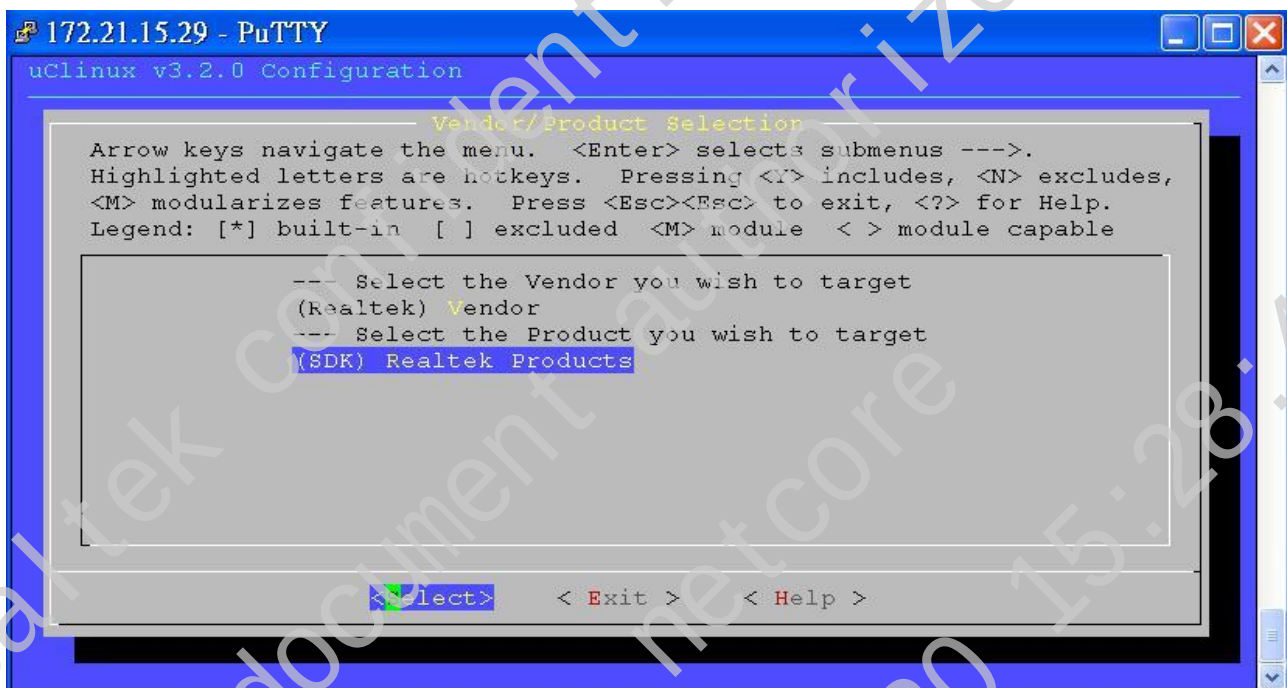


Figure 4.3-1

- (1) romfs.subdirs:  
In this target, the CONFIG\_LINUXDIR will be used to modified the “X.X.XX” string of rc file, then install rc into romfs.
- (2) linux linux%\_only:  
For Linux 2.6.32.58, the “-I\$(SDKINCCOM)” should be added, then SDK include files could be referenced during making kernel image.
- (3) pre\_setup:  
If the \$(CONFIG\_PRODUCT) is “SDK”, the target.sys.run.setup should not be included.
- (4) config.in:  
If the \$(CONFIG\_PRODUCT) is “SDK”, the target.sys.vendor.build

- should not be included.
- (5) romfs:  
If the `$(CONFIG_PRODUCT)` is “SDK”, the `target.sys.install` should not be included.  
This target will build user space applications.
  - (6) clean:  
If the `$(CONFIG_PRODUCT)` is “SDK”, the `target.sys.clean` should not be included.
  - (7) real\_clean mrproper:  
If the `$(CONFIG_PRODUCT)` is “SDK”, the `$(SYS_CFGDIR)` should not be cleaned.
  - (8) distclean:  
If the `$(CONFIG_PRODUCT)` is “SDK”, the `target.sys.cleanup` should not be included.
  - (9) `$(SYSROMFSDIR)`  
If the `$(CONFIG_PRODUCT)` is “SDK”, this parameter should be configured as `$(ROMFSDIR)`.
  - (10) `$(TURNKEY_)` related parameters  
If the `$(CONFIG_PRODUCT)` is “SDK”, those `$(TURNKEY_)` related parameters should be configured.

#### 4.4 Makefile in /lib folder

There are some targets are modified in this Makefile.

- (1) all:  
For uClibc-0.9.28, the make operations will be executed.  
For uClibc-0.9.30, there are nothing need to do.
- (2) romfs:  
For uClibc-0.9.28, it will use the make to install libraries.  
For uClibc-0.9.30, it will copy libraries to romfs directly.

#### 4.5 Pre-Created Symbolic Links

By Realtek’s BSP Makefile architecture, during menuconfig, the `config.in` should be referenced, so these default symbolic links should be existed at first. There are two major symbolic links is pre-created in trunk for Linux BSP part.

- (1) target (/uClinux folder):  
It links to /sdk/system/linux/linux-2.6.32.x/arch/mips/realtek/sel\_chip.
- (2) sel\_chip:  
By default, sel\_chip links to ./boards/rtl8328.  
Once more chips should be supported in the future, the sel\_chip symbolic link should be modified by configuration.

## 4.6 SDK Kernel Module Makefile

In original SDK kernel module Makefiles, some compile options are added in CFLAGS, and some options are added in EXTRA\_CFLAGS. The adding rule is not consistent.

Under the Linux kernel Makefile architecture, the EXTRA\_CFLAGS will impact current Makefile only, it will not pollute other Makefiles. Therefore, all CFLAGS are changed to EXTRA\_CFLAGS for SDK kernel modules' Makefiles.

The following Makefiles are modified.

- (1) Makefile.linux.drivers
- (2) Makefile.linux.kernel.rtcore
- (3) Makefile.linux.kernel.rtdrv
- (4) Makefile.linux.kernel.rtk
- (5) Makefile.linux.kernel.rtnic
- (6) Makefile.linux.kernel.ut
- (7) Makefile.linux.net

## 4.7 SDK Source code

Some SDK source codes are modified to fulfill new Linux kernel. All the kernel version related source codes are separated by the following kind of compile flag. By the way, the <linux/version.h> should be included in source codes.

```
#if (LINUX_VERSION_CODE > KERNEL_VERSION(2,6,19))
 XXXXX XXXX XXXXX
#endif
```

## Chapter 5 U-Boot Makefile Modification

### 5.1 config.mk in /u-boot/cpu/mips/ folder

In msdk, the “-mcpu=4kec” should not be used.

### 5.2 config.mk in /u-boot/ folder

In msdk, the “-nostdinc” should be used.

Moreover, this file should include the PRJDIR/kernel/uClinux/.config.

### 5.3 Makefile in /u-boot/ folder

The \$(SDKSYSDIR) should be configured as current path base on Linux kernel version.

Moreover, this file should include the PRJDIR/kernel/uClinux/.config.

Once the MIPS34Kc chip is selected such as RTL8390, and the msdk tool chain is used, then the “-mmt” should be added in CFLAGS.

## Chapter 6 User Folder Makefile Modification

### 6.1 Makefile in /kernel/uClinux/user/busybox/

Add \$(CONFIG\_PRODUCT) to decide the turnkey configuration files should be included or not.

### 6.2 Makefile in /kernel/uClinux/user/switch/

Add \$(CONFIG\_PRODUCT) to decide the turnkey configuration files should be included or not.

---

**Realtek Semiconductor Corp.**

**Headquarters**

No. 2, Innovation Road II

Hsinchu Science Park, Hsinchu 300, Taiwan

Tel.: +886-3-578-0211. Fax: +886-3-577-6047

[www.realtek.com](http://www.realtek.com)